技术部第二次技术分享会

Java 基础语法・简明版

姚雨森　技术部分管主席

2026-10-23

让程序主动“开口”

我们需要知道程序正在执行什么，可以让它把信息打印到控制台。

使用 System.out.println(...) 输出一行文字：

```
System.out.println("Hello, Java!");
```

双引号里面的文字会被打印出来，双引号本身不会显示。

控制台输出：

```
Hello, Java!
```

把输出语句写在 main 中

程序从 main 方法开始执行。先把输出语句写在它的大括号内：

```
public class Main {
    public static void main(String[] args) {
        System.out.println("First");
        System.out.println("Second");
    }
}
```

这里的两条语句**从上到下依次执行**，每次输出后换行：

```
First
Second
```

注释：给人看的说明

注释用于解释代码，注释中的内容不会被当作语句执行。

- 单行注释：`//` 后面直到本行末尾的内容。
- 多行注释：`/*` 与 `*/` 之间的内容，可以跨行。

下面的代码写在 `main` 中：

```
// Print a greeting.
System.out.println("Hello!"); // This is a comment.
/* The next statement is commented out.
System.out.println("Hidden");
*/
System.out.println("Goodbye!");
```

控制台依次输出 `Hello!` 和 `Goodbye!`，各占一行；`Hidden` 不会输出。

变量：保存程序需要的数据

程序有时需要做运算，也需要保存一些数据，比如数值、文本。
可以用**变量**保存这些数据，再通过变量名读取或修改它们。

变量的声明与赋值

声明变量时赋值（初始化）

语法：　类型　变量名　=　初始值;

```
int price = 60;
```

也可以先声明，再赋值

声明：　类型　变量名;
赋值：　变量名　=　值;

```
int price;
price = 60;
System.out.println(price);
```

赋值时不用再写类型。

数据类型：不同信息有不同表达

商品数量、单价、是否有货和商品名，需要表达不同性质的信息。

要保存什么	常用类型	值的例子
整数数量	`int`	`2`
带小数的数值	`double`	`12.5`
是或否	`boolean`	`true`
单个字符	`char`	`'A'`
一段文本	`String`	`"Notebook"`

先想清楚数据是什么，再选择类型。

boolean：表达“是或否”

商品有没有货？用户是不是会员？这些状态只有两种答案。

```
boolean inStock = true;
boolean isMember = false;

System.out.println(inStock);  // true
System.out.println(isMember); // false
```

- `true` 表示真，`false` 表示假。
- `boolean` 只有这两个值，不用数字 0 和 1 代替。

后面可以根据这些状态，让程序采取不同的动作。

字符与字符串：保存文字

一个等级字母可以用 char，商品名称可以用 String。

```
char grade = 'A';
String name = "Notebook";

System.out.println(name);   // Notebook
System.out.println("name"); // name
```

- 字符用单引号，如 'A'；文本用双引号，如 "Notebook"。
- name 读取变量的值，"name" 表示文字本身。
- String 首字母大写，不属于基本数据类型。

字符串拼接：让结果看得懂

只看到一个数字还不够，我们可以给结果加上说明。

```
String name = "Notebook";
double total = 25.0;
System.out.println(name + " total: " + total);
```

输出： Notebook total: 25.0

算术运算符：加、减、乘、除与取余

我们可以对数字做运算，把结果保存到变量中，或直接打印出来。

符号	运算	表达式示例	结果
+	加法	7 + 2	9
-	减法	7 - 2	5
*	乘法	7 * 2	14
/	除法	7 / 2	3
%	取余	7 % 2	1

```
int sum = 7 + 2;
System.out.println(sum);   // 9
```

% 求除法的余数，常被称为“取模”；这里先看正整数。

比较运算：结果是 boolean

比较两个数值，结果是 `true` 或 `false`，可以保存到 `boolean` 变量中。

运算符	含义	示例	结果
`==`	等于	`7 == 2`	`false`
`!=`	不等于	`7 != 2`	`true`
`>`	大于	`7 > 2`	`true`
`<`	小于	`7 < 2`	`false`
`>=`	大于或等于	`7 >= 7`	`true`
`<=`	小于或等于	`7 <= 2`	`false`

```
boolean passed = 80 >= 60;
System.out.println(passed); // true
```

注意：`=` 是赋值，`==` 才是判断是否相等。

逻辑运算：组合或取反条件

逻辑运算的操作数和结果都是 `boolean`。

运算符	含义	结果为 true 的条件
`&&`	与	两边都为 `true`
`||`	或	至少一边为 `true`
`!`	非（取反）	原来的值为 `false`

```
int score = 80;
boolean inRange = (score >= 0) && (score <= 100); // true
boolean outside = (score < 0) || (score > 100);   // false
boolean failed = !(score >= 60);                  // false
```

综合示例：商品总价计算需求

购买 A、B 两种商品，根据各自的单价和数量计算总价。

- 用 `priceA`、`priceB` 保存两种商品的单价，单位为元。
- 用 `quantityA`、`quantityB` 保存各自的购买数量。
- 直接修改代码中四个变量的初始值。
- 计算总价，并在控制台打印结果。

计算公式：总价 = A 的单价 × 数量 + B 的单价 × 数量。

示例数据：A 每件 12.5 元，买 2 件；B 每件 8 元，买 3 件。

综合示例：商品总价计算实现

```
public class Main {
    public static void main(String[] args) {
        double priceA = 12.5;
        double priceB = 8.0;
        int quantityA = 2;
        int quantityB = 3;
        double total = priceA * quantityA + priceB * quantityB;
        System.out.println("Total: " + total);
    }
}
```

- 先分别计算两种商品的金额，再相加得到总价。
- 本例输出 `Total: 49.0`，即总价为 49 元。

新需求：满足条件时才打折

现在增加一个需求：总价**超过 40 元**时打八折。
也就是：当 `total > 40` 时，让 `total` 乘以 0.8；否则保持不变。

if：满足条件时才执行

if 语法：条件为 true 时执行，否则跳过。

```
if（条件）{
    条件成立时执行的语句
}
```

条件必须是 boolean。例如：总价超过 40 元时打八折。

```
double total = 49.0;
if (total > 40) {
    total = total * 0.8;
}
System.out.println("Total: " + total); // Total: 39.2
```

新需求：打折与未打折时输出不同文字

如果打折了，输出文字也要变化：

- 打折时，输出 `Discounted total:` 和折后总价。
- 未打折时，输出 `Total:` 和正常总价。

原来的写法只在 `if` 中修改价格，最后总会执行外面的同一条输出语句，因而无法区分这两种文字。

自然的想法是：**如果打折了，就按折扣文本打印；否则，按正常文本打印。**

用 `else` 表示“否则”，让程序在两条分支中选择一条执行。

if-else：在两条分支中选择一条

条件为 `true` 时执行 `if` 分支，为 `false` 时执行 `else` 分支。

语法

```
if (条件) {
    条件成立时执行
} else {
    条件不成立时执行
}
```

折扣示例

```
double total = 49.0;
if (total > 40) {
    total = total * 0.8;
    System.out.println(
        "Discounted total: " + total);
} else {
    System.out.println(
        "Total: " + total);
}
```

新需求：按总价划分价格档次

假设总价非负，我们希望根据金额输出对应的价格档次。

总价范围	档次
$0 \le \texttt{total} \le 40$	便宜（Cheap）
$40 < \texttt{total} \le 100$	适中（Moderate）
$\texttt{total} > 100$	昂贵（Expensive）

40 元属于“便宜”，100 元属于“适中”，每个总价只属于一个档次。

先用三个独立的 if 实现

```
double total = 80.0;
if (total <= 40) {
    System.out.println("Cheap");
}
if (total > 40 && total <= 100) {
    System.out.println("Moderate");
}
if (total > 100) {
    System.out.println("Expensive");
}
```

这三个判断在逻辑上是一组：对**同一个不变的总价**，恰好只有一个条件成立。但三个独立的 `if` 仍会从上到下，逐个判断条件。

修改总价后，后面的 if 可能再次成立

假设超过 100 元打五折，并把这个判断放在另外两个判断之间：

```
double total = 180.0;
if (total <= 40) {
    System.out.println("Cheap");
}
if (total > 100) {
    total = total * 0.5;
    System.out.println("Discounted");
}
if (total > 40 && total <= 100) {
    System.out.println("Moderate");
}
```

180 元变成 90 元后，又满足“适中”条件，两个分支都执行了。

else if：一组条件，只执行一个分支

我们需要把多个判断连成一组：**一旦某个条件成立，后面的条件就不再判断。**

```
if (条件 1) {
    条件 1 成立时执行的语句
} else if (条件 2) {
    条件 2 成立时执行的语句
} else {
    以上条件都不成立时执行的语句
}
```

- 从上到下判断，只执行**第一个成立条件**对应的分支。
- 即使分支中修改了变量，也不会继续判断后面的 `else if`。
- 最后的 `else` 可以省略；省略后可能一个分支也不执行。

新需求：余额不足时充值

商品总价 180 元，余额只有 50 元。假设每次充值增加 50 元。

先用 `if`：余额不足，就充值一次。

```
double total = 180.0;
double balance = 50.0;
if (balance < total) {
    balance = balance + 50;
}
System.out.println("Balance: " + balance); // 100.0
```

问题：充值一次后，余额仍然不够付款。

`if` 只判断一次，不会自动回头检查余额是否已经足够。

充值后重新检查，直到余额足够

我们需要的是：**只要余额不足，就继续充值。**

1. 检查余额是否小于商品总价。
2. 如果不足，充值 50 元，然后回到第一步。
3. 如果已经足够，结束充值，继续付款。

本例余额变化：$50 \to 100 \to 150 \to 200$ 元。

余额达到 200 元时，不再满足“余额不足”，于是停止充值。

用 `while` 表示这种**条件成立就反复执行**的过程。

while：条件成立就重复执行

每轮先判断条件：为 `true` 时执行循环体，再回到条件处；为 `false` 时退出。

语法

```
while（条件）{
    反复执行的语句
}
```

条件必须是 `boolean`。

充值与付款

```
double total = 180.0;
double balance = 50.0;
while (balance < total) {
    balance = balance + 50;
}
balance = balance - total;
System.out.println(
    "Remaining: " + balance); // 20.0
```

- 一开始余额就足够时，循环一次也不执行，直接付款。
- 如果余额不足，却不在循环中增加余额，就会一直循环。

用 while 计数：打印商品编号

新需求：依次打印第 1～5 件商品的编号。

```
int number = 1;
while (number <= 5) {
    System.out.println("Item: " + number);
    number++;
}
```

- 初始化：`int number = 1;`，从 1 开始。
- 继续条件：`number <= 5`，编号不超过 5。
- 更新：`number++`，每轮结束后加 1。

三个部分分散在不同位置；如果忘记 `number++`，就会一直打印编号 1。

for：把计数循环的三个部分写在一起

for 把初始化、继续条件和更新集中到循环开头，便于检查。

```
for（初始化；继续条件；更新）{
    每轮执行的语句
}
```

同样打印第 1～5 件商品的编号：

```
for (int number = 1; number <= 5; number++) {
    System.out.println("Item: " + number);
}
```

- 初始化只执行一次；每轮先判断条件，再执行循环体。
- 循环体结束后执行更新，再回到条件处；条件为假时结束。
- for 仍需写更新语句，并不会自动帮我们加 1。

遇到问题时，想到这些工具

我想做什么	可以用什么
看见程序的结果	输出
保存和更新数据	变量
表达不同性质的信息	数据类型
计算数值、组织文字	运算、字符串拼接
根据情况执行不同操作	分支
重复执行一段操作	循环

先想清楚数据、条件和重复过程，再查具体写法。

接下来需要自主学习的内容

- **（必选）**Java 基础语法完整版 PPT
- **（可选）**《Java 核心技术卷 I》第三章

课后练习

1. **（必选）**完成补充语句的练习项目。

项目链接之后会发到群里。